Case Study 5: Advanced Numeric Pattern Generator (Hollow Diamond)

Activity sheet case study #27 | Concepts: Strings, nested loops, pattern generation, conditional statements, input validation, lists

1. Problem Statement

Produce a hollow diamond made of numbers for a size n entered by the user. Row numbers appear only on the two edges of the diamond. The program must validate the input and keep the output perfectly aligned.

2. Objectives

- Use nested loops to build a pattern one character position at a time.
- Control alignment with arithmetic on column positions.
- Validate string input before converting it to an integer.
- Generate a symmetric pattern from an ascending and a descending row sequence.

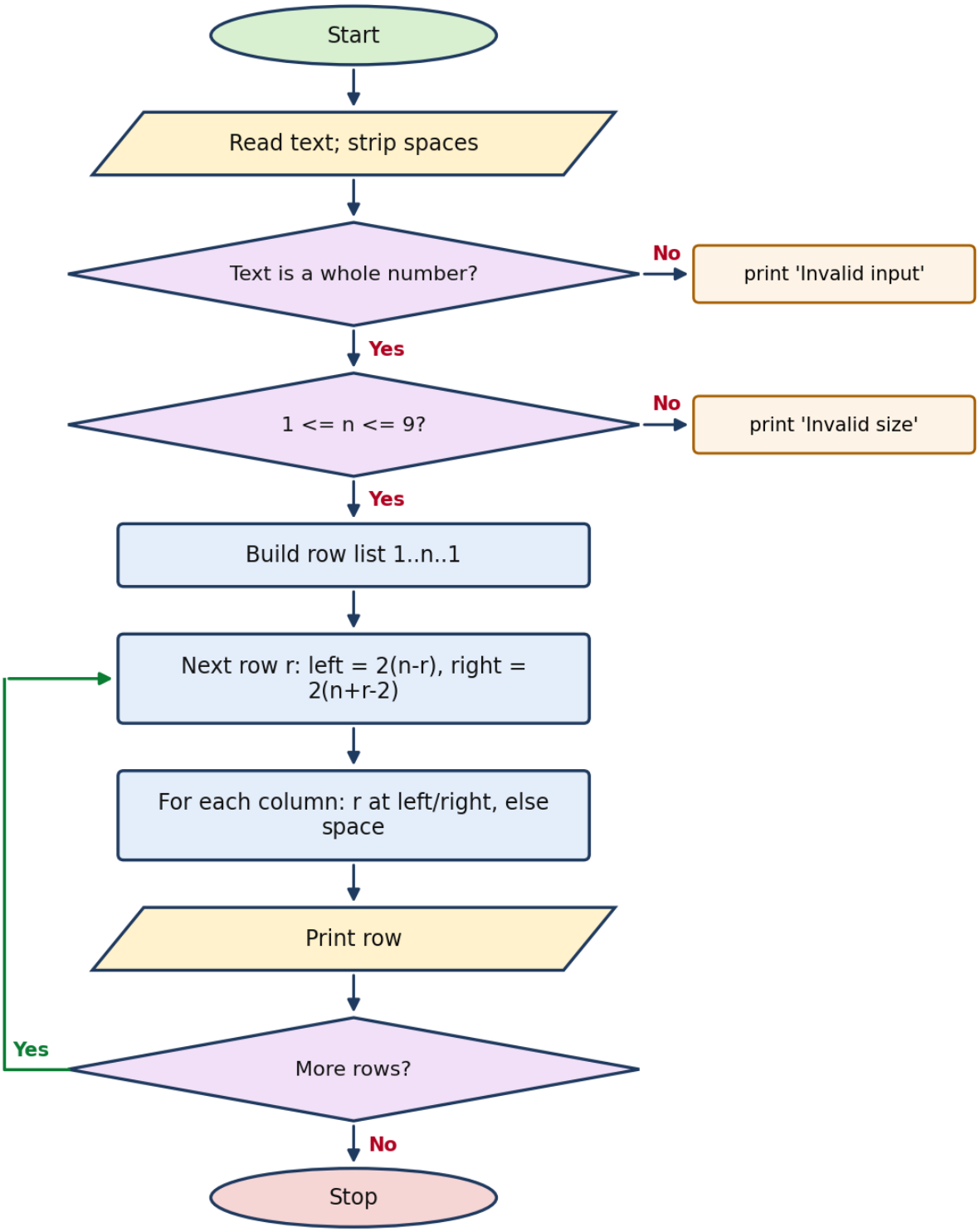
3. Assumptions

- n is the number of rows in the upper half, so the diamond has 2n - 1 rows.
- Valid sizes are whole numbers from 1 to 9 so that every row number is a single digit and the layout stays aligned.
- Each cell is one character wide; the row number r is printed at column 2(n - r) on the left edge and 2(n + r - 2) on the right edge.

4. Algorithm

1. Read the input as text and strip spaces.
2. If the text is not a whole number print an error and stop.
3. Convert to n; if n < 1 or n > 9 print a range error and stop.
4. Build the list of row numbers: 1, 2, ..., n, n-1, ..., 1.
5. For each row number r compute the left column and right column.
6. Loop over all columns 0 .. 4n-4; print r at the left or right column, otherwise a space.
7. Print the row without trailing spaces and continue to the next row.

5. Flowchart



6. Pseudocode

```
START
  READ text ; text <- strip(text)
  IF text is not all digits THEN PRINT "Invalid input" ; STOP
  n <- INTEGER(text)
  IF n < 1 OR n > 9 THEN PRINT "Invalid size" ; STOP
  width <- 4*n - 3
  rows <- [1, 2, ..., n, n-1, ..., 1]
  FOR each r in rows
```

```
        left  <- 2*(n - r)
        right <- 2*(n + r - 2)
        line  <- ""
        FOR col = 0 TO width-1
            IF col = left OR col = right THEN line <- line + r
            ELSE line <- line + " "
        PRINT line without trailing spaces
    END FOR
STOP
```

7. Python Source Code

```
# Case Study 27: Hollow Numeric Diamond
text = input("Enter diamond size n (1 to 9): ").strip()

if not text.isdecimal():
    print("Invalid input: please enter a whole number such as 5.")
else:
    n = int(text)
    if n < 1 or n > 9:
        print("Invalid size: n must be between 1 and 9.")
    else:
        width = 4 * n - 3

        # rows 1, 2, ..., n, n-1, ..., 1
        row_numbers = []
        for i in range(1, n + 1):
            row_numbers.append(i)
        for i in range(n - 1, 0, -1):
            row_numbers.append(i)

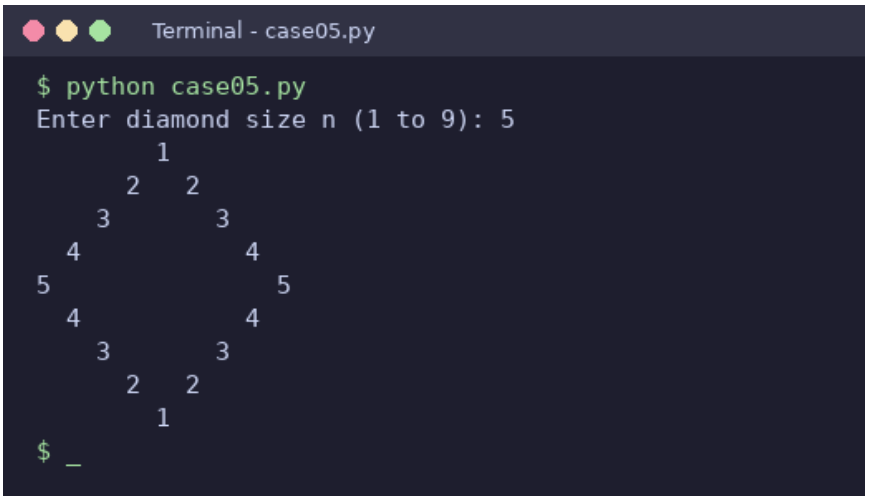
        for r in row_numbers:
            left = (n - r) * 2
            right = (n + r - 2) * 2
            line = ""
            for col in range(width):
                if col == left or col == right:
                    line = line + str(r)
                else:
                    line = line + " "
            print(line.rstrip())
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Typical size	5	9 aligned rows; widest row has 15 spaces inside	`4 4 | 5 5`	**PASS**
2	Small size	3	Row '3 3' and apex ' 1'	`3 3 | 1`	**PASS**
3	Minimum size	1	Single digit 1	`Enter diamond size n (1 to 9): 1`	**PASS**
4	Maximum size	9	Widest row for n = 9	`9 9`	**PASS**
5	Zero	0	Range error	`Invalid size: n must be between 1 and 9.`	**PASS**
6	Too large	12	Range error	`Invalid size: n must be between 1 and 9.`	**PASS**
7	Negative number	-3	Not a whole number	`Invalid input: please enter a whole number such as 5.`	**PASS**
8	Text input	abc	Not a whole number	`Invalid input: please enter a whole number such as 5.`	**PASS**
9	Decimal and blank	5.5	Not a whole number	`Invalid input: please enter a whole number such as 5.`	**PASS**
10	Input with spaces	4	Spaces stripped; valid	`4 4`	**PASS**

9. Output Screenshot

Terminal output of `python case05.py` using the default data:



10. Conclusion

The generator builds each row column by column with nested loops, which makes alignment depend only on simple arithmetic. Input validation prevents crashes for text, negative, decimal and out-of-range values, and the symmetric row list avoids writing the pattern logic twice.